GRU 게이트 순환 유닛 (Gated Recurrent Unit)

문 두 개, 기억 하나로 줄인 LSTM

Machine Learning and Deep Learning

KSA 2027-Spring
Sumin Han (suminhan@ksa.hs.kr)

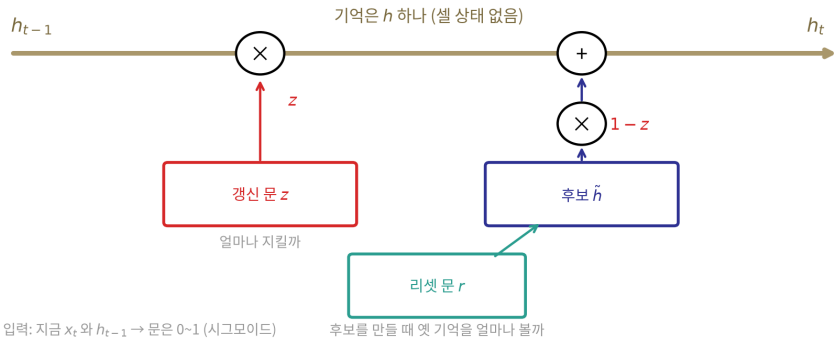
오늘의 질문: 문 세 개, 기억 두 개, 다 필요할까?

	LSTM (1997, 2000)	?
문	잊기 f · 적기 i · 꺼내기 o **3개**	**2개**
기억	셀 상태 c + 출력 h **2개**	h **하나**
가중치 묶음	4묶음 (RNN 의 4배)	**3묶음**

잊는 만큼 적으면 되지 않을까? f 와 i 를 **하나로** 묶자

Cho 외 (2014): 다음 시간 Seq2Seq 의 바로 그 번역 논문에서 함께 나왔다

GRU: 지키는 만큼 덜 적는다



갱신 문 z: 옛 기억을 z 만큼 지키고, 나머지 $1-z$ 만큼 새 후보로 채운다

리셋 문 r: 후보를 만들 때 옛 기억을 얼마나 참고할지

식으로

$$\mathbf{z}_t = \sigma(W_z[\mathbf{h}_{t-1}, \mathbf{x}_t]) \qquad \mathbf{r}_t = \sigma(W_r[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

$$\tilde{\mathbf{h}}_t = \tanh(W_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t])$$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t$$

σ: $0 \sim 1$ 칸마다 따로 정한다 $\odot$ = 칸별 곱 편향은 생략

방향 주의 원 논문 · PyTorch 와 같은 방향 (z = 지키는 비율). 반대로 쓰는 책도 있다

손으로 한 번: 갱신 문

이전 기억 $h_{t-1} = 0.8$ 후보 $\tilde{h} = -0.2$ 갱신 문 $z = 0.9$

$$h_t = \underbrace{0.9 \times 0.8}_{\text{keep 90\%}} + \underbrace{0.1 \times (-0.2)}_{\text{write 10\%}} = 0.72 - 0.02 = \mathbf{0.70}$$

	지키기	적기	
LSTM (지난 시간 예)	$f = 0.9$	$i = 0.5$	(따로 정한다)
GRU	$z = 0.9$	$1 - z = \mathbf{0.1}$	(자동으로 정해진다)

90% 를 지키면 새 내용은 10% 만 들어간다 h 는 늘 옛 기억과 후보의 **가중 평균**

손으로 한 번: 리셋 문

후보 $\tilde{h} = \tanh(\, w\,x + u\,(r\,h_{t-1})\,)$ $\quad x = 1,\ w = 0.5,\ u = 2,\ h_{t-1} = 0.8$

$r = 1$ $\quad \tanh(0.5 + 1.6) = \tanh 2.1 \quad = 0.97$ 옛 기억이 섞인 후보

$r = 0$ $\quad \tanh(0.5 + 0) = \tanh 0.5 \quad = \mathbf{0.46}$ 지금 입력만 본 후보

$r \approx 0$: “지난 내용은 잊고 새로 시작” 예: 새 문장, 새 단어의 첫 글자

리셋 문은 후보만 바꾼다. 옛 기억을 실제로 얼마나 버릴지는 여전히 z 가 정한다

LSTM 과 나란히

	LSTM	GRU
지키기	잊는 문 f	갱신 문 z
적기	적는 문 i (따로)	$1 - z$ **(f 와 i 를 묶음)**
꺼내기	꺼내는 문 o	없음: 기억을 **그대로** 보여 준다
후보 만들 때	이전 출력 h 를 다 본다	리셋 문 r 만큼만 본다
기억	c (안) + h (밖)	h 하나
상태 넘기기	(h, c)	h

왜 오래 기억할 수 있나

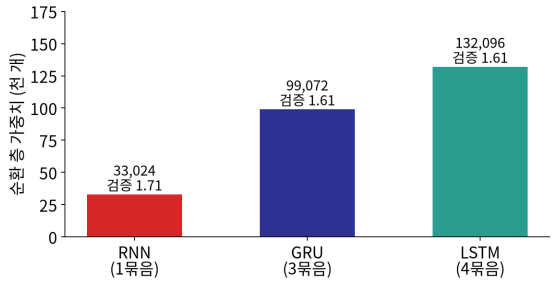
$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + \cdots \quad \Rightarrow \quad \text{지름길로 } \mathbf{z}_t \text{ 만 곱해진다}$$

LSTM 의 $c_t = f \odot c_{t-1} + \cdots$ 와 같은 **더하기** 경로 행렬도 tanh 도 없다

기억해야 할 때 $z \approx 1$ 로 학습하면 $0.999^{100} \approx$ **90%** 기울기도 그대로 전달

기본 RNN: 걸음마다 W_h 와 $\tanh'$ 을 곱한다 $\rightarrow 0.9^{100} \approx 0.003\%$

손으로 세기: 가중치는 LSTM 의 $\frac{3}{4}$

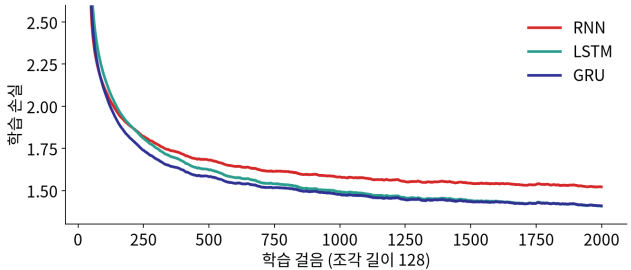


묶음 하나 (입력 x 칸, 은닉 h 칸):
$h \times (x + h)$ + 편향

RNN	1묶음
GRU	**3묶음**: z, r, 후보
LSTM	4묶음: f, i, o, 후보

입력 128 · 은닉 128, PyTorch (편향 2개):
$3 \times 33{,}024 = 99{,}072$

실험: 셰익스피어 글자 모델, RNN vs LSTM vs GRU



검증 손실 (뒤 10% 글)	조각 64	조각 128	가중치
RNN	1.74	1.71	3.3만
LSTM	**1.64**	**1.61**	13.2만
GRU	1.66	**1.61**	9.9만

LSTM 과 GRU, 무엇을 쓸까?

Chung 외 (2014)	음악 · 음성: GRU $\approx$ LSTM, 둘 다 기본 RNN 보다 낫다
Jozefowicz 외 (2015)	구조 1만 개 넘게 비교: GRU 가 앞선 과제가 많았지만, 잊는 문 편향을 1 로 둔 LSTM 이 거의 따라잡았다
우리 실험	검증 손실 같음 (1.61), 가중치는 GRU 가 $\frac{3}{4}$

정답은 없다: 데이터가 적거나 빨라야 하면 GRU, 아니면 둘 다 해 보기

PyTorch: `nn.LSTM` $\leftrightarrow$ `nn.GRU` 한 줄 바꾸기 (상태만 `(h, c)` $\rightarrow$ `h`)

문제풀이

1. $h_{t-1} = 0.6, \tilde{h} = -0.2, z = 0.75$ 이면 h_t 는? $z = 0$ 이면?
2. 갱신 문이 계속 0.99 면 100걸음 뒤 옛 기억은 몇 % 남나? ($0.99^{100} \approx 0.366$)
3. 입력 100칸, 은닉 128칸인 GRU 층의 가중치 수는? (3묶음, 편향 하나로 세기) 같은 크기 LSTM 과 차이는?
4. GRU 에는 꺼내는 문이 없다. 그래도 “기억은 하되 지금 출력에는 드러내지 않기”를 하려면?

실습

1. 갱신 문 · 리셋 문 손 계산 0.70 · 0.46, numpy GRU 한 칸 → `nn.GRUCell` 과 비교
2. 셰익스피어 글자 모델: `nn.LSTM` vs `nn.GRU` (손실 · 가중치 · 한 걸음 시간)
3. 학습된 GRU 의 갱신 문 들여다보기 → 글 써 보기

Tiny Shakespeare (Karpathy, char-rnn)

▶ Colab에서 실습 노트북 열기

정리

GRU (Cho 외 2014): 문 두 개, 기억 h 하나

$h_t = z \odot h_{t-1} + (1 - z) \odot \tilde{h}$ 지키는 만큼 덜 적는다 (f 와 i 를 묶음)

리셋 문 r: 후보를 만들 때 옛 기억을 얼마나 볼지 꺼내는 문은 없다

가중치 LSTM 의 $\frac{3}{4}$ 셰익스피어 검증 손실 1.61 (LSTM 과 같음) 다음: Seq2Seq